



Covering All the Bases: Type-Based Verification of Test Input Generators

ZHE ZHOU, Purdue University, USA

ASHISH MISHRA, Purdue University, USA

BENJAMIN DELAWARE, Purdue University, USA

SURESH JAGANNATHAN, Purdue University, USA

Test input generators are an important part of property-based testing (PBT) frameworks. Because PBT is intended to test deep semantic and structural properties of a program, the outputs produced by these generators can be complex data structures, constrained to satisfy properties the developer believes is most relevant to testing the function of interest. An important feature expected of these generators is that they be capable of producing *all* acceptable elements that satisfy the function's input type and generator-provided constraints. However, it is not readily apparent how we might validate whether a particular generator's output satisfies this *coverage* requirement. Typically, developers must rely on manual inspection and post-mortem analysis of test runs to determine if the generator is providing sufficient coverage; these approaches are error-prone and difficult to scale as generators become more complex. To address this important concern, we present a new refinement type-based verification procedure for validating the coverage provided by input test generators, based on a novel interpretation of types that embeds "*must-style*" underapproximate reasoning principles as a fundamental part of the type system. The types associated with expressions now capture the set of values *guaranteed* to be produced by the expression, rather than the typical formulation that uses types to represent the set of values an expression *may* produce. Beyond formalizing the notion of *coverage types* in the context of a rich core language with higher-order procedures and inductive datatypes, we also present a detailed evaluation study to justify the utility of our ideas.

CCS Concepts: • **Software and its engineering** → **Language types; Verification and validation.**

Additional Key Words and Phrases: refinement types, property-based testing, underapproximate reasoning

ACM Reference Format:

Zhe Zhou, Ashish Mishra, Benjamin Delaware, and Suresh Jagannathan. 2023. Covering All the Bases: Type-Based Verification of Test Input Generators. *Proc. ACM Program. Lang.* 7, PLDI, Article 157 (June 2023), 24 pages. <https://doi.org/10.1145/3591271>

1 INTRODUCTION

Property-based testing (PBT) is a popular technique for automatically testing deep semantic and structural properties of programs. Originally pioneered by the QuickCheck [Claessen 2020] library for Haskell, PBT frameworks now exist for many programming languages, including JavaScript [FastCheck 2022], Rust [RustCheck 2021], Python [Hypothesis 2022], Scala [ScalaCheck 2021], and Coq [Lampropoulos and Pierce 2022]. The PBT methodology rests on two key components: *executable properties* that capture the expected input-output behaviors of the program under test, and *test input generators* that generate random values of the input types needed to validate

Authors' addresses: Zhe Zhou, Purdue University, USA, zhou956@purdue.edu; Ashish Mishra, Purdue University, USA, mishr115@purdue.edu; Benjamin Delaware, Purdue University, USA, bendy@purdue.edu; Suresh Jagannathan, Purdue University, USA, suresh@cs.purdue.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2023 Copyright held by the owner/author(s).

2475-1421/2023/6-ART157

<https://doi.org/10.1145/3591271>

these behaviors. In contrast to unit tests, which rely on single examples of inputs and outputs, generators are meant to provide a *family* of inputs against which programs can be tested, with the goal of ensuring the set of generated tests provide good coverage of all possible inputs. In order to prune out irrelevant inputs, PBT frameworks allow users to define custom generators that reflect the specific shape of data that the developer believes is most likely to trigger interesting (aka faulty) behavior. As one simple example, to test a tree compression or balancing function, the developer may want to use a generator that produces n -ary trees with randomly chosen height and arity but whose leaves are ordered according to a user-provided ordering relation.

Given the critical role they play in the assurance case provided by PBT frameworks, it is reasonable to ask what constitutes a “good” specification for a test generator. For our example, one answer could be that it should only produce ordered trees. Of course, this is not a very satisfactory characterization of the behavior we desire: the “constant” generator that always produces trees of height one trivially meets this specification, but it is unlikely to produce useful tests for a compression function! Ideally, we would like a generator to intelligently enumerate the space of *all* possible ordered trees, thereby helping to maximize the likelihood of finding bugs in the function under test. Because defining such an enumeration procedure for arbitrary datatypes can be hard, even when complete enumeration is computationally feasible, PBT frameworks instead give developers the ability to assemble generators for complex data structures *compositionally*, building on generators for simpler types where randomly sampling elements of the type is straightforward and sufficient. For example, we could implement an ordered tree generator in terms of a primitive random number generator that is used to non-deterministically select the height, arity, and elements of a candidate tree, checking (or enforcing) the orderliness of the tree before returning it as a feasible test input. Although the random number generator might provide a guarantee that its underlying probability density function (PDF) is always non-zero on all elements in its sample space, determining that a tree generator that is built using it can actually enumerate all the ordered trees desired is a substantially harder problem. Even if we know the generator is capable of eventually yielding all trees, constraints imposed by the function’s precondition might require the generator to perform further filtering or transformations over generated trees. However, proving that any filtering operations the generator uses do not mistakenly prune out valid ordered trees or that any transformations the generator performs over candidate trees preserve the elements of the random tree being transformed, pose additional challenges. In other words, verifying that the generator is *complete* with respect to our desired orderliness property entails reasoning that is independent of the behavior of the primitive generators used to build the tree. Consequently, we require some alternative mechanism to help qualify the part of the target function’s input space the generator is actually guaranteed to cover. Devising such a mechanism is challenging precisely because the properties that need to be tested may impose complex structural and semantic constraints on the generated output (e.g., requiring that an output tree be a binary search tree, or that it satisfies a red-black property, etc.); the complexity of these constraints is directly correlated to the sparseness of the function’s input space preconditions.

```

1 type 'a tree =
2   | Leaf
3   | Node of ('a * 'a tree * 'a tree)
4 let rec bst_gen (lo: int) (hi: int) : int tree =
5   if lo + 1 >= hi then Leaf else
6   (* Leaf ⊕ *)
7   (let (x: int) = int_range (lo + 1, hi - 1) in
8    Node (x, bst_gen lo x, bst_gen x hi))

```

Fig. 1. A BST generator. Failing to uncomment line 6 results in the generator never producing trees that contain only a subset of the elements in the interval between lo and hi , which is inconsistent with the developer’s intent.

To illustrate this distinction more concretely, consider the input test generator shown in Figure 1 that is intended to generate all binary search trees (BSTs) whose elements are between the interval

`lo` and `hi`. If we ignore the comment on line 6, we can conclude this generator always produces a non-empty BST whenever $lo < hi$. While the generator is correct - it always generates a well-formed BST - it is also incomplete; the call `bst_gen 0 10`, for example, will never produce a tree containing just `Leaf` or a tree with a shape like `Node(1, Leaf, Leaf)`, even though these instances are valid trees consistent with the constraints imposed by the generator's argument bounds. In fact, this implementation *never* generates a BST that only contains a proper subset of the elements that reside within the interval defined by `lo` and `hi`. By uncommenting line 6, however, we allow the generator to non-deterministically choose (via operator $\oplus$) to either return a `Leaf` or left and right BST subtrees based on value returned by the `int_range` generator, enabling it to potentially produce BSTs containing all valid subsets of the provided interval, thus satisfying our desired completeness behavior. The subtleties involved in reasoning about such coverage properties is clearly non-trivial. We reiterate that recognizing the distinction between these two implementations is not merely a matter of providing a precise output type capturing the desired sortedness property of a BST: the incomplete implementation clearly satisfies such a type! Furthermore, simply knowing that the underlying `int_range` generator used in the implementation samples all elements within the range of the arguments it is provided is also insufficient to conclude that the BST generator can yield *all* possible BSTs within the supplied interval. Similar observations have led prior work to consider ways to improve a generator's coverage through mechanisms such as fuzzing [Dolan 2022; Lampropoulos et al. 2019], or to automatically generate complete-by-construction generators for certain classes of datatypes [Lampropoulos et al. 2018].

In contrast to these approaches, this paper embeds the notion of coverage as an integral part of a test input generator's *type* specification. By doing so, a generator's type now specifies the set of behaviors the generator is *guaranteed* to exhibit; a well-typed generator is thus guaranteed to produce *every possible* value satisfying a desired structural property, e.g., that the repaired (complete) version of `bst_gen` is capable of producing every valid BST. By framing the notion of coverage in type-theoretic terms, our approach neither requires instrumentation of the target program to assess the coverage effectiveness of a candidate generator (as in Lampropoulos et al. [2019]) nor does it depend on a specific compilation strategy for producing generators (as in Lampropoulos et al. [2018]). Instead, our approach can automatically verify the coverage properties of an *arbitrary* test input generator, regardless of whether it was hand-written or automatically synthesized.

Key to our approach is a novel formulation of a *must*-style analysis [Germane and McCarthy 2021; Godefroid et al. 2010; Jagannathan et al. 1998] of a test input generator's behavior in type-theoretic terms. In our proposed type system, we say an expression e has *coverage type* τ if every value contained in τ *must* be producible by e . Note how this definition differs from our usual notion of what a type represents: ordinarily, if e has type τ then we are allowed to conclude only that any value contained in τ *may* be produced by e . Informally, types interpreted in this usual way define an *overapproximation* of the values an expression e can yield, without obligating e to produce any specific such value. In contrast, coverage types define an *underapproximation* - they characterize the values an expression e has to produce, potentially eliding other values that e may also evaluate to. When the set of elements denoted by a generator's (underapproximate) coverage type matches that of its (overapproximate) normal type, however, we can soundly assert that the generator is complete. As we illustrate in the remainder of the paper, this characterization allows us to reason about a program's coverage behavior on the same formal footing as its safety properties.

In this sense, our solution can be seen a type-theoretic interpretation of recently proposed Incorrectness Logics (IL) [Le et al. 2022; O'Hearn 2019; Raad et al. 2020], in much the same way that refinement-type systems like Liquid Types [Jhala and Vazou 2021; Vazou et al. 2014] relate to traditional program logics [Hoare 1969]. Despite the philosophical similarities with IL, however, we use underapproximate reasoning for a very different goal. While IL has been primarily used to

precisely capture the conditions that will lead a program to fault, this work explores how type-based underapproximate reasoning can be used to verify the completeness properties of a test generator in the context of PBT.

This interpretation leads to a fundamental recasting of how types relate to one another: ordinarily, we are always allowed to assert that $\tau <: \top$. This means that any typing context that admits an expression with type τ can also admit that expression at a type with a logically weaker structure. In contrast, the subtyping relation for coverage types inverts this relation, so that $\top <: \tau$. Intuitively, $\top$ represents the coverage type that obligates an expression ascribed this type to be capable of producing *all* elements in τ . But, any context that requires an expression to produce all such elements can always guarantee that the expression will also produce a subset of these elements. In other words, we are always allowed to weaken an overapproximation (i.e., grow the set of values an expression may evaluate to), and strengthen an underapproximation (i.e., shrink the set of values an expression must evaluate to). Thus, in our setting, a random number generator over the integers has coverage type $\top_{\text{int}}$ under the mild assumption that its underlying PDF provides a non-zero likelihood of returning every integer. In contrast, a faulty computation like $1 \div 0$ has coverage type $\perp$ since there are no guarantees provided by the computation on the value(s) it must return. Here, $\perp$ represents a type that defines a degenerate underapproximation, imposing no constraints on the values an expression ascribed this type must produce.

This paper makes the following contributions:

- (1) It introduces the notion of *coverage types*, types that characterize the values an input test generator is guaranteed to (i.e., *must*) yield.
- (2) It formalizes the semantics of coverage types in an ML-like functional language with support for higher-order functions and inductive datatypes.¹
- (3) It develops a bi-directional type-checking algorithm for coverage types in this language.
- (4) It incorporates these ideas in a tool (Poirot) that operates over OCaml programs equipped with input generators and typed using coverage types, and presents an extensive empirical evaluation justifying their utility, by verifying the coverage properties of both hand-written and automatically synthesized generators for a rich class of datatypes and their structural properties.

The remainder of the paper is structured as follows. In the next section, we present an informal overview of the key features of our type system. Section 3 presents the syntax and semantics for a core call-by-value higher-order functional language with inductive datatypes that we use to formalize our approach. Section 4 describes our coverage type system, and its metatheory is given in Section 5. A bidirectional typing algorithm is then given in Section 6. We describe details about the implementation of Poirot and provide benchmark results in Section 7. Related work and conclusions are given in Sections 8 and 9.

2 OVERVIEW

Before presenting the full details of our type system, we begin with an informal overview of its key features.

Base types. In the following, we write $[v:b \mid \phi]$ to denote the coverage type that qualifies the base type b using the predicate ϕ . As described in the previous section, an application of the primitive built-in generator for random numbers: $\text{int_gen} : \text{unit} \rightarrow \text{int}$ has the coverage type $\text{int_gen}() : [v:\text{int} \mid \top_{\text{int}}]$. We use brackets $[...]$ to emphasize that a coverage type has a different meaning from the types typically found in other refinement type systems [Jhala and Vazou 2021;

¹A Coq formalization of this calculus, its type system, and its metatheory is provided on Zenodo [Zhou et al. 2023b].

Table 1. Examples of overapproximate and underapproximate (coverage) typings. We use $\vdash$ and $\nvdash$ to identify whether a term can or cannot be assigned the corresponding type, resp. The constant `err` represents a special error value, which causes the program to halt when encountered.

<code>int_gen ()</code>	$\vdash [v:int \mid \top_{int}]$ $\vdash [v:int \mid \top_{int}]$	$\vdash [v:int \mid v = 1 \vee 2]$ $\nvdash [v:int \mid v = 1 \vee 2]$	$\vdash [v:int \mid v = 1]$ $\nvdash [v:int \mid v = 1]$	$\vdash [v:int \mid \perp]$ $\nvdash [v:int \mid \perp]$
<code>1</code>	$\vdash [v:int \mid v = 1]$ $\vdash [v:int \mid v = 1]$	$\vdash [v:int \mid \perp]$ $\nvdash [v:int \mid \top_{int}]$	$\vdash [v:int \mid \top_{int}]$ $\nvdash [v:int \mid v = 1 \vee 2]$	$\vdash [v:int \mid v = 1 \vee 2]$ $\nvdash [v:int \mid \perp]$
<code>err</code>	$\vdash [v:int \mid \perp]$ $\nvdash [v:int \mid \top_{int}]$	$\nvdash [v:int \mid \top_{int}]$ $\nvdash [v:int \mid v = 1 \vee 2]$	$\nvdash [v:int \mid v = 1 \vee 2]$ $\nvdash [v:int \mid v = 1]$	$\nvdash [v:int \mid v = 1]$ $\nvdash [v:int \mid \perp]$

Vazou et al. 2014] where a qualified type b , written as $\{v:b \mid \phi\}$, uses a predicate ϕ to constrain the set of values a program *might* evaluate to. To illustrate this distinction, consider the combinations of expressions and types shown in Table 1. These examples demonstrate the previous observation that it is always possible to strengthen the refinement predicate used in an underapproximate type and weaken such a predicate in an overapproximate type. A similar phenomena appears in IL’s rule of consequence, which inverts the direction of the implications on pre- and postconditions in the overapproximate version of the rule. As a result, the bottom type $[v:int \mid \perp]$ is the universal supertype in our type hierarchy, as it places no restrictions on the values a term *must* produce. Thus, we sometimes abbreviate $[v:int \mid \perp]$ as `int`, since the information provided by both types is the same. Importantly, the coverage type for the error term (`err`) can *only* be qualified with $\perp$, since an erroneous computation is unconstrained with the respect to the values it is obligated to produce.

Coverage types can also qualify inductive datatypes, like lists and trees. In particular, the complete generator for BSTs presented in the introduction can be successfully type-checked using the following result type:

$$[v:int \text{ tree} \mid bst(v) \wedge \forall u, mem(v, u) \implies lo < u < hi]$$

where $bst(v)$ and $mem(v, u)$ are *method predicates*, i.e., uninterpreted functions used to encode semantic properties of the datatype. In the type given above, the qualifier requires that bst_gen ’s result is a BST (encoded by the predicate $bst(v)$) and that every element u stored in the tree (encoded by the predicate $mem(v, u)$) is between lo and hi ; the coverage type thus constrains the implementation to produce *all* trees that satisfy this qualifier predicate. In contrast, the incomplete version of the generator (i.e., the implementation that does not allow prematurely terminating tree generation with a Leaf node) could only be type-checked using the following (stronger) type:

$$[v:int \text{ tree} \mid bst(v) \wedge \forall u, mem(v, u) \iff lo < u < hi]$$

This signature asserts that all trees produced by the generator are BSTs, that any element contained in the tree is within the interval bounded by lo and hi , *and* moreover, any element in that interval *must* be included in the tree. The subtle difference between the two implementations, reflected in the different implication constraints expressed in their respective refinements, precisely captures how their coverage properties differ.

Control Flow. Just as underapproximate coverage types invert the standard overapproximate subtyping relationship, they also invert the standard relationship between a control flow construct and its subexpressions. To see how, consider the simple generator for even numbers shown in Figure 2. When the integer generator, `int_gen()`, yields an odd number, `even_gen` faults; otherwise it simply returns the generated number. Consider the following type judgment that arises when type checking this program:

$$n:[v:int \mid \top_{int}], b:[v:bool \mid v \iff n \bmod 2 = 0] \vdash \text{if } b \text{ then } n \text{ else } err : [v:int \mid v \bmod 2 = 0] \quad (1)$$

```

let even_gen () =
  let (n: int) = int_gen () in
  let (b: bool) = n mod 2 == 0 in
  if b then n else err

```

Fig. 2. An even number generator defined in terms of an integer number generator.

Intuitively, this judgment asserts that the `if` expression covers all even numbers (i.e., has the type $[v:int \mid v \bmod 2 = 0]$) assuming that the *local variable* n can be instantiated with an arbitrary number, and that the variable b is true precisely when n is even. Notice how the typing context encodes the potential control-flow path that *must* reach the non-faulting branch of the conditional expression. Enforcing the requirement that the conditional be able to return all even numbers does *not* require each of its branches to be a subtype of the expected type, in contrast to standard type systems. Our type system must instead establish that, *in total*, the values produced by each of the branches cover the even numbers. Because the false branch of the conditional faults, it is only typeable at the universal supertype, i.e., $[v:int \mid \perp]$. Thus, if the standard subtyping relationship between this conditional and its branches held, it could only be typed at $[v:int \mid \perp]$! This is not the case in our setting, as the true branch contributes all the desired outputs. Formally, this property is checked by the following assumption of the coverage typing rule for conditionals:

$$n:[v:int \mid \top_{\text{int}}], b:[v:bool \mid v \iff n \bmod 2 = 0] \vdash [v:int \mid (b \wedge v = n) \vee (\neg b \wedge \perp)] <: [v:int \mid v \bmod 2 == 0]$$

The $b \wedge v = n$ and $\neg b \wedge \perp$ subformulas correspond to the types of the true and false branches,² respectively. Taking the disjunction of these two formulas describes the set of values that can be produced by either branch;³ this subtyping relationship guarantees this type is at least as large as the type expected by the entire conditional. To check that this subtyping relationship holds, our type checker generates the following formula:

$$\forall v, (v \bmod 2 = 0) \implies (\exists n, \top \wedge \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge v = n) \vee (\neg b \wedge \perp)) \quad (2)$$

This formula aligns with the intuitive meaning of (1): in our type system, coverage types of variables in the typing context tell us what values they must (at least) produce. When checking whether a particular subtyping or typing relationship holds, we are free to choose *any* instantiation of the variables that entails the desired property. Accordingly, in (2), the variables n and b are *existentially* quantified to indicate there exists an execution path that instantiates these local variables in a way that produces the output v , instead of being universally quantified as they would be in a standard refinement type system.

Function types. To type functions, most refinement type systems add a restricted form of the *dependent function types* found in full-spectrum dependent type systems. Such types allow the qualifiers in the result type of a function to refer to its parameters, enabling the expression of rich safety conditions governing the arguments that may be supplied to the function. To see how this capability might be useful in our setting, consider the test generator `bst_gen` from the introduction. The complete version of this function produces all BSTs whose elements fall between the range specified by its two parameters, `lo` and `hi`. For the bounds 0 and 3, the application `bst_gen 0 3` can be typed as: $[v:int \mid \text{bst}(v) \wedge \forall u, \text{mem}(v, u) \implies 0 < u < 3]$. Using the standard typing rule for functions, the only way to encode this relationship in the type of `bst_gen` is:

$$[v:int \mid v = 0] \rightarrow [v:int \mid v = 3] \rightarrow [v:int \text{ tree} \mid \text{bst}(v) \wedge \forall u, \text{mem}(v, u) \implies 0 < u < 3]$$

Of course, this specification fails to account for the behaviors of `bst_gen` when supplied with different bounds: for example, the application `bst_gen 2 7` will fail to typecheck against this type.

Since the desired coverage property of `bst_gen` fundamentally depends on the kinds of inputs given to it, our type system includes dependent products of the form:

$$\text{lo}:[v:int \mid \top_{\text{int}}] \rightarrow \text{hi}:[v:int \mid \text{lo} \leq v] \rightarrow [v:int \text{ tree} \mid \text{bst}(v) \wedge \forall u, \text{mem}(v, u) \implies \text{lo} < u < \text{hi}]$$

We use the notation $\{\dots\}$ to emphasize that the argument types of a dependent arrow have a similar purpose and interpretation as in standard refinement type systems. Thus, the above type can be

²As is standard in dependent type systems, the types of both branches have been refined to reflect the path conditions under which they will be executed.

³This is similar to how the derived rule of choice in IL uses disjunction to reason about both branches of a nondeterministic choice statement.

read as “if the inputs lo and hi are *any* number such that $lo \leq hi$, then the output *must* cover all possible BSTs whose elements are between lo and hi ”. Using this type for `bst_gen` allows our system to seamlessly type-check both `(bst_gen 0 3)` and `(bst_gen 2 7)`. Our typing algorithm will furthermore flag the call `(bst_gen 3 1)` as being ill-typed, since the function’s type dictates that the generator’s second argument (1) *may* only be greater than or equal to its first (3).

```
let bst_gen_low_bound (low: int) =  
  let (high: int) = int_gen () in  
  bst_gen low high
```

Fig. 3. This function generates a BST with a supplied lower bound, `low`.

Function Application. Since the type of a function parameter is interpreted as a normal (overapproximate, “may”) refinement type, while arguments in an application may be typed using (underapproximate, “must”) coverage types, we need to be able to bridge the gap between may and must types when typing function applications. Intuitively, our type system does

so by ensuring that the set of values in the coverage type of the argument has a nonempty overlap with the set of possible values expected by the function. We establish this connection by using the fact that the typing context captures the control flow paths that may and must exist when the function is called. To illustrate this intuition concretely, consider the function `bst_gen_low_bound` shown in Figure 3. This function generates all non-empty BSTs whose elements are integers with the lower bound given by its parameter. The judgment we need to check is of the form:

$$\text{bst_gen} : lo : \{v : \text{int} \mid \tau_{\text{int}}\} \rightarrow hi : \{v : \text{int} \mid lo \leq v\} \rightarrow [v : \text{int tree} \mid \dots], \text{low} : \{v : \text{int} \mid \tau_{\text{int}}\}, \text{high} : [v : \text{int} \mid \tau_{\text{int}}] \\ \vdash \text{bst_gen low high} : \dots$$

Note that the type for `low` is a normal refinement type that specifies a safety condition for function `bst_gen_low_bound`, namely that `low` *may* be any number. In contrast, the type for `high` is a coverage type, representing the result of `int_gen()` that indicates that it *must* (i.e., guaranteed to) be any possible integer. However, the signature for `bst_gen` demands that parameter `hi` only be supplied values greater than its first argument (`lo`); we incorporate this requirement by *strengthening* `high`’s type (via a subsumption rule) to reflect this additional constraint when typing the body of the `let` expression in which `high` is bound. This strengthening, which is tantamount to a more refined underapproximation, allows us to typecheck the application `(bst_gen low high)` in the following context:

$$\text{bst_gen} : lo : \{v : \text{int} \mid \tau_{\text{int}}\} \rightarrow hi : \{v : \text{int} \mid lo \leq v\} \rightarrow [v : \text{int tree} \mid \dots], \text{low} : \{v : \text{int} \mid \tau_{\text{int}}\}, \text{high} : [v : \text{int} \mid lo \leq v]$$

The coverage type associated with `high` guarantees that `int_gen()` must produce values greater than `low` (along with possibly other values). To ensure that the result type of the call reflects the underapproximate (coverage) dependences that exist between `low` and `high`, we introduce existential quantifiers in the type’s qualifier:

$$\dots, \text{low} : \{v : \text{int} \mid \tau_{\text{int}}\} \vdash [v : \text{int tree} \mid \text{bst}(v) \wedge \exists \text{high}, \text{low} \leq \text{high} \wedge \forall u, \text{mem}(v, u) \implies \text{low} < u < \text{high}]$$

This type properly captures the behavior of the generator: it is guaranteed to generate all BSTs characterized by a lower bound given `low` such that there exists an upper bound `high` where $\text{low} \leq \text{high}$ and in which every element in the tree is contained within these bounds.

Summary. Coverage types invert many of the expected relationships that are found in a normal refinement type system. Here, qualifiers provide an *underapproximation* of the values that an expression may evaluate to, in contrast to the typically provided *overapproximation*. This, in turn, causes the subtyping relation to invert the standard relationship entailed by logical implication between type qualifiers. Our coverage analysis also considers the *disjunction* of the coverage guarantees provided by the branches of control-flow constructs, instead of their *conjunction*. Finally, when applying a function with a dependent arrow type to a coverage type, we check semantic inclusion between the overapproximate and underapproximate constraints provided by the two types, and manifest the paths that witness the elements guaranteed to be produced by the coverage type through existentially-quantified variables in the application’s result type.

Variables	$x, f, u, \dots$
Data constructors	$d ::= () \mid \text{true} \mid \text{false} \mid \text{O} \mid \text{S} \mid \text{Cons} \mid \text{Nil} \mid \text{Leaf} \mid \text{Node}$
Constants	$c ::= \mathbb{B} \mid \mathbb{N} \mid \mathbb{Z} \mid \dots \mid d \bar{c}$
Operators	$op ::= d \mid + \mid == \mid < \mid \text{mod} \mid \text{nat_gen} \mid \text{int_gen} \mid \dots$
Values	$v ::= c \mid op \mid x \mid \lambda x:t.e \mid \text{fix } f:t.\lambda x:t.e$
Terms	$e ::= v \mid \text{err} \mid \text{let } x = e \text{ in } e \mid \text{let } x = op \bar{v} \text{ in } e \mid \text{let } x = v \text{ in } e$ $\mid \text{match } v \text{ with } \overline{d \bar{y}} \rightarrow e$
Base Types	$b ::= \text{unit} \mid \text{bool} \mid \text{nat} \mid \text{int} \mid b \text{ list} \mid b \text{ tree} \mid \dots$
Basic Types	$t ::= b \mid t \rightarrow t$
Method Predicates	$mp ::= \text{emp} \mid \text{hd} \mid \text{mem} \mid \dots$
Literals	$l ::= c \mid x$
Propositions	$\phi ::= l \mid \perp \mid \top_b \mid op(\bar{l}) \mid mp(\bar{x}) \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \implies \phi \mid \forall u:b. \phi \mid \exists u:b. \phi$
Refinement Types	$\tau ::= [v:b \mid \phi] \mid \{v:b \mid \phi\} \mid x:\tau \rightarrow \tau$
Type Contexts	$\Gamma ::= \emptyset \mid \Gamma, x:\tau$

Fig. 4. λ^{TG} syntax.

3 LANGUAGE

Terms. In order to formalize our typed-based verification approach of input test generators, we introduce a core calculus for test generators, λ^{TG} . The language, whose syntax is summarized in Figure 4, is a call-by-value lambda-calculus with pattern-matching, inductive datatypes, and well-founded (i.e., terminating) recursive functions whose argument must be structurally decreasing in all recursive calls made in the function's body. The syntax of λ^{TG} is expressed in monadic normal-form (MNF) [Hatchliff and Danvy 1994], a variant of A-Normal Form (ANF) [Flanagan et al. 1993] that allows nested let-bindings. The language additionally allows faulty programs to be expressed using the error term `err`. As discussed in Section 2, this term is important in our investigation because coverage types capture an expression's reachability properties, and we need to ensure the guarantees offered by such types are robust even in the presence of stuck computations induced by statements like `err`. The language is also equipped with primitive operators to generate natural numbers, integers, etc. (`nat_gen` (), `int_gen` (), ...) that can be used to express various kinds of non-deterministic behavior relevant to test input generation. As an example, the $\oplus$ choice operator used in Figure 1 can be defined as:

$$e_1 \oplus e_2 \doteq \text{let } n = \text{nat_gen} () \text{ mod } 2 \text{ in match } n \text{ with } 0 \rightarrow e_1 \mid _ \rightarrow e_2$$

Note that the primitive generators of λ^{TG} are completely agnostic to the specific sampling strategy they employ, as long as they ensure every value in their range has a nonzero likelihood of being generated. Indeed, λ^{TG} does not include any operators to bias the frequency at which values are produced, e.g., QuickCheck's frequency. The inclusion of such an operator would not change anything fundamentally about our type system or its guarantees. λ^{TG} has a completely standard small-step operational semantics.

Types. Like other refinement type systems [Jhala and Vazou 2021], λ^{TG} supports three classes of types: *base types*, *basic types*, and *refinement types*. Base types (b) include primitive types such as `unit`, `bool`, `nat`, etc., and inductive datatypes (e.g., `int list`, `bool tree`, `int list list`,

Well-Formedness

$$\boxed{\Gamma \vdash^{\text{WF}} \tau}$$

$$\frac{\Gamma \equiv x_i:\{v:b_{x_i} \mid \phi_{x_i}\}, y_j:[v:b_{y_j} \mid \phi_{y_j}], z:(a:\tau_a \rightarrow \tau_b) \quad (\forall x_i:b_{x_i}, \exists y_j:b_{y_j}, \forall v:b, \phi) \text{ is a Boolean predicate} \quad \forall j, \text{err} \notin \llbracket [v:b_{y_j} \mid \phi_{y_j}] \rrbracket_{\Gamma}}{\Gamma \vdash^{\text{WF}} [v:b \mid \phi]} \text{ WFBASE}$$

$$\frac{\Gamma, x:\{v:b \mid \phi\} \vdash^{\text{WF}} \tau}{\Gamma \vdash^{\text{WF}} x:\{v:b \mid \phi\} \rightarrow \tau} \text{ WFBARG} \quad \frac{\Gamma \vdash^{\text{WF}} (a:\tau_a \rightarrow \tau_b) \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash^{\text{WF}} (a:\tau_a \rightarrow \tau_b) \rightarrow \tau} \text{ WFBRES}$$

Subtyping

$$\boxed{\Gamma \vdash \tau_1 <: \tau_2}$$

$$\frac{\llbracket [v:b \mid \phi_1] \rrbracket_{\Gamma} \subseteq \llbracket [v:b \mid \phi_2] \rrbracket_{\Gamma}}{\Gamma \vdash [v:b \mid \phi_1] <: [v:b \mid \phi_2]} \text{ SUBUBASE} \quad \frac{\llbracket \{v:b \mid \phi_1\} \rrbracket_{\Gamma} \subseteq \llbracket \{v:b \mid \phi_2\} \rrbracket_{\Gamma}}{\Gamma \vdash \{v:b \mid \phi_1\} <: \{v:b \mid \phi_2\}} \text{ SUBOBASE}$$

$$\frac{\Gamma \vdash \tau_{21} <: \tau_{11} \quad \Gamma, x:\tau_{21} \vdash \tau_{12} <: \tau_{22}}{\Gamma \vdash x:\tau_{11} \rightarrow \tau_{12} <: x:\tau_{21} \rightarrow \tau_{22}} \text{ SUBARR}$$

Disjunction

$$\boxed{\Gamma \vdash \tau_1 \vee \tau_2 = \tau_3}$$

$$\frac{\llbracket \tau_1 \rrbracket_{\Gamma} \cap \llbracket \tau_2 \rrbracket_{\Gamma} = \llbracket \tau_3 \rrbracket_{\Gamma}}{\Gamma \vdash \tau_1 \vee \tau_2 = \tau_3} \text{ DISJUNCTION}$$

Fig. 5. Auxillary typing relations

etc.). Basic types (t) extend base types with function types. Refinement types (τ) qualify base types with both underapproximate and overapproximate propositions, expressed as predicates defined in first-order logic (FOL). Function parameters can also be qualified with overapproximate refinements that specify when it is safe to apply this function. In contrast, the return type of a function can only be qualified using an underapproximate refinement, reflecting the coverage property of the function's result and thus characterizing the values the function is guaranteed to produce. The erasure of a type τ , $[\tau]$, is the type that results from erasing all qualifiers in τ .

Qualifiers. To express rich shape properties over inductive datatypes, we allow propositions to reference method predicates, as it is straightforward to generate verification conditions using these uninterpreted functions that can be handled by an off-the-shelf theorem prover like Z3 [de Moura and Bjørner 2008]. As we describe in Section 6, our typechecking algorithm imposes additional constraints on the form propositions can take, in order to ensure that its validity is decidable. In particular, we ensure that Z3 queries generated by our typechecker to check refinement validity are always over effectively propositional (EPR) sentences (i.e., prenex-quantified formulae of the form $\exists^* \forall^* \varphi$ where φ is quantifier-free.)

4 TYPE SYSTEM

Despite superficial similarities to other contemporary type systems [Jhala and Vazou 2021], the typing rules⁴ of λ^{TG} differ in significant ways from those of its peers, due to the fundamental semantic distinction that arises when viewing types as an underapproximation and not overapproximation of program behavior.

⁴The full set of typing rules (including the basic typing rules and the bidirectional typing rules from Section 6), proofs of theorems, and the details of our evaluation are provided in full version of the paper [Zhou et al. 2023a].

Table 2. Example typings for λ^{TG} primitives

Constants	$\text{Ty}(\text{true}) = [v:\text{bool} \mid v]$ $\text{Ty}(8) = [v:\text{nat} \mid v = 8] \dots$
Data Constructors	$\text{Ty}([]) = [v:b \text{ list} \mid \text{emp}(v)]$ $\text{Ty}(\text{Cons}) = x:\{v:b \mid \tau_b\} \rightarrow y:\{v:b \text{ list} \mid \tau_{\text{list}}\} \rightarrow [v:b \text{ list} \mid \text{hd}(v, x) \wedge \text{tl}(v, y)] \dots$
Operators	$\text{Ty}(\text{nat_gen}) = \{v:\text{unit} \mid \tau_{\text{unit}}\} \rightarrow [v:\text{nat} \mid \tau_{\text{nat}}]$ $\text{Ty}(+) = x:\{v:\text{nat} \mid \tau_{\text{unit}}\} \rightarrow y:\{v:\text{nat} \mid \tau_{\text{nat}}\} \rightarrow [v:\text{nat} \mid v = x + y] \dots$

Our type system depends on three auxiliary relations shown in Figure 5. The first group defines well-formedness conditions on a type under a particular type context, i.e., a sequence of variable-type bindings consisting of overapproximate refinement types, underapproximate coverage types, and arrow (function) types. A type τ that is well-formed under a type context Γ needs to meet three criteria: (1) the qualifier in τ needs to be closed in the current typing context, and the denotation⁵ of all the coverage types ($[v:b_{y_j} \mid \phi_{y_j}]$) found in Γ should not include err (WFBASE); (2) overapproximate types may only appear in the domain of a function type (WFARG); and, (3) underapproximate coverage types may only appear in the range of a function type (WFRES). To understand the motivation for the first criterion, observe that a type context in our setting provides a witness to feasible execution paths in the form of bindings to local variables. Accordingly, *no* type is well formed under the type context $x:[v:\text{nat} \mid \perp]$ or under $x:\{v:\text{nat} \mid v > 0\}$, $y:[v:\text{nat} \mid x = 0 \wedge v = 2]$, as neither context corresponds to a valid manifest execution path. On the other hand, a well-formed type is allowed to include an error term in its denotation, e.g., type $[v:\text{nat} \mid \perp]$ is well-formed under type context $x:\{v:\text{nat} \mid v > 0\}$ as it always corresponds to a valid underapproximation.

Our second set of judgments defines a largely standard subtyping relation based on the underlying denotation of the types being related. Note also that over- and under-approximate types are incomparable— our typing rules tightly control when one can be treated as another.

The disjunction rule (DISJUNCTION), which was informally introduced in Section 2, merges the coverage types found along distinct control paths. Intuitively, the type $[v:\text{nat} \mid v = 1 \vee v = 2]$ is the disjunction of the types $[v:\text{nat} \mid v = 1]$ and $[v:\text{nat} \mid v = 2]$. Notice that only an inhabitant of both $[v:\text{nat} \mid v = 1]$ and $[v:\text{nat} \mid v = 2]$ should be included in their disjunction: e.g., the term $1 \oplus 2$ is one such inhabitant. Thus, we formally define this relation as the *intersection* of the denotations of two types.

The salient rules of our type system are defined in Figure 6. All our typing rules assume that all terms are well-typed according to the normal (aka non-refined) type system. The rules collectively maintain the invariant that terms can only be assigned a well-formed type. The rule for constants (TCONST) is straightforward. It relies on an auxiliary function, Ty , to assign types to the primitives of λ^{TG} . Table 2 presents some examples of the typings provided by Ty . We use method predicates in the types of constructors: the types for list constructors, for example, use emp , hd and tl , to precisely capture that $[]$ constructs an empty list, and that $(\text{Cons } x \ y)$ builds a list containing x as its head and y as its tail.⁶

The typing rules for function abstraction (TFUN) and error (TERR) are similarly straightforward. The type of the function's argument τ needs to be consistent with the type of the argument's erasure ($\lfloor \tau_x \rfloor$) specified by the λ -abstraction. The error term can be assigned an arbitrary bottom coverage base type. The variable rule (TVARBASE) establishes that the variable x in the type context with a base type can also be typed with the tautological qualifier $v = x$ (the rule's well-formedness assumption ensures that x is not free). This judgment allows us to, for example, type the function $\lambda x : \text{nat}.x$ with the type $x:\{v:\text{nat} \mid \tau_{\text{nat}}\} \rightarrow [v:\text{nat} \mid v = x]$, indicating that the return value is guaranteed to be exactly equal to the input x . Observe that the type of x under the type context $x:\{v:\text{nat} \mid \tau_{\text{nat}}\}$

⁵The definition of a type's denotation is given in Section 5.

⁶The auxiliary function Ty also provides a type for operators, thus the rule for operators is the same as TCONST.

Typing

$$\boxed{\Gamma \vdash e : \tau}$$

$$\begin{array}{c}
\frac{\Gamma \vdash^{\text{WF}} \text{Ty}(c)}{\Gamma \vdash c : \text{Ty}(c)} \text{ TCONST} \quad \frac{\Gamma, x:\tau_x \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} x:\tau_x \rightarrow \tau}{\Gamma \vdash \lambda x: [\tau_x]. e : (x:\tau_x \rightarrow \tau)} \text{ TFUN} \quad \frac{\Gamma \vdash^{\text{WF}} [v:b \mid \perp]}{\Gamma \vdash \text{err} : [v:b \mid \perp]} \text{ TERR} \\
\\
\frac{\Gamma \vdash^{\text{WF}} [v:b \mid v = x]}{\Gamma \vdash x : [v:b \mid v = x]} \text{ TVARBASE} \quad \frac{\Gamma(x) = (a:\tau_a \rightarrow \tau_b) \quad \Gamma \vdash^{\text{WF}} a:\tau_a \rightarrow \tau_b}{\Gamma \vdash x : (a:\tau_a \rightarrow \tau_b)} \text{ TVARFUN} \\
\\
\frac{\Gamma \vdash v_1 : a:\{v:b \mid \phi\} \rightarrow \tau_x \quad \Gamma \vdash v_2 : [v:b \mid \phi] \quad \Gamma, x:\tau_x [a \mapsto v_2] \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash \text{let } x = v_1 \ v_2 \text{ in } e : \tau} \text{ TAPP} \quad \frac{\emptyset \vdash \tau <: \tau' \quad \emptyset \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash e : \tau'} \text{ TSUB} \quad \frac{\Gamma \vdash \tau' <: \tau \quad \Gamma \vdash \tau <: \tau' \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash e : \tau'} \text{ TEQ} \\
\\
\frac{\Gamma \vdash v : \tau_v \quad \Gamma \vdash^{\text{WF}} \tau \quad \Gamma, \bar{y}:\bar{\tau}_y \vdash d_i(\bar{y}) : \tau_v \quad \Gamma, \bar{y}:\bar{\tau}_y \vdash e_i : \tau}{\Gamma \vdash (\text{match } v \text{ with } \bar{d}_i \bar{y} \rightarrow e_i) : \tau} \text{ TMATCH} \quad \frac{\Gamma \vdash e : \tau_1 \quad \Gamma \vdash e : \tau_2 \quad \Gamma \vdash \tau_1 \vee \tau_2 = \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash e : \tau} \text{ TMERGE} \\
\\
\frac{\Gamma \vdash \lambda x: b. \lambda f: (b \rightarrow [\tau]). e : (x:\{v:b \mid \phi\} \rightarrow f:(x:\{v:b \mid v < x \wedge \phi\} \rightarrow \tau) \rightarrow \tau) \quad \Gamma \vdash^{\text{WF}} x:\{v:b \mid \phi\} \rightarrow \tau}{\Gamma \vdash \text{fix } f: (b \rightarrow [\tau]). \lambda x: b. e : (x:\{v:b \mid \phi\} \rightarrow \tau)} \text{ TFix}
\end{array}$$

Fig. 6. Selected typing rules

(generated by the function rule TFUN) is *not* $[v:\text{nat} \mid \tau_{\text{nat}}]$. We cannot simply duplicate the qualifier for x from the type context here, as this is only sound when types characterize an overapproximation of program behavior. As an example, $\{v:\text{nat} \mid \tau_{\text{nat}}\}$ is a subtype of $\{v:\text{nat} \mid v = x\}$ under the type context $x:\{v:\text{nat} \mid \tau_{\text{nat}}\}$; in our underapproximate coverage type system, in contrast, $[v:\text{nat} \mid \tau_{\text{nat}}]$ is not a subtype of $[v:\text{nat} \mid v = x]$ under the type context $x:\{v:\text{nat} \mid \tau_{\text{nat}}\}$.

The typing rule for application (TAPP) requires both its underapproximate argument type and the overapproximate parameter type to have the same qualifier, and furthermore requires that the type of the body (τ) is well-formed under the original type context Γ , enforcing that x (the result of the application) does not appear free in τ . When argument and parameter qualifiers are not identical, a subsumption rule is typically used to bring the two types into alignment. Recall the following example from Section 2, suitably modified to conform to λ^{TG} 's syntax:

```

bst_gen : lo:{v:int |  $\tau_{\text{int}}$ }  $\rightarrow$  hi:{v:int |  $\{lo \leq v\} \rightarrow [v:\text{int tree} \mid \dots]$ }, low : {v:int |  $\tau_{\text{int}}$ }  $\vdash$ 
  let (g: unit  $\rightarrow$  int) = int_gen in let (x: unit) = () in
  let (high: int) = g x in let (y: int tree) = bst_gen low high in y

```

Here, the type of high, $[v:\text{int} \mid \tau_{\text{int}}]$ is stronger than the type expected for the second parameter of `bst_gen`, $[v:\text{int} \mid lo \leq v]$. The subsumption rule (TSUB), that would normally allow us to strengthen the type of high to align with the required parameter type, is applicable to only closed terms, which high is not. For the same reason, we cannot use TSUB to strengthen the type of high when it is bound to `g x`. Thankfully, we can strengthen `g` when it is bound to `int_gen`: according to Table 2, the operator `int_gen` has type $\{v:\text{unit} \mid \tau_{\text{unit}}\} \rightarrow [v:\text{int} \mid \tau_{\text{int}}]$ and is also closed, and can thus be strengthened via TSUB, allowing us to type the call to `bst_gen` under the following, stronger type context:

```

bst_gen : lo:{v:int |  $\tau_{\text{int}}$ }  $\rightarrow$  hi:{v:int |  $\text{lo} \leq v$ }  $\rightarrow$  [v:int tree | ...], low : {v:int |  $\tau_{\text{int}}$ },
g : {v:unit |  $\tau_{\text{unit}}$ }  $\rightarrow$  [v:int |  $\text{low} \leq v$ ], x : [v:unit |  $\tau_{\text{unit}}$ ], high : [v:int |  $\text{low} \leq v$ ]  $\vdash$ 
  let (y: int tree) = bst_gen low high in y

```

The subsumption rule allows us to use `int_gen` in a context that requires *fewer* guarantees than `int_gen` actually provides, namely those values of `high` required by the signature of `bst_gen`. Intuitively, since our notion of coverage types records feasible executions in the type context in the form of existentials that serve as witnesses to an underapproximation, the strengthening provided by the subsumption rule establishes an invariant that all bindings introduced into a type context only characterize valid behaviors in a program execution. When coupled with `TMERGE`, this allows us to *split* a typing derivation into multiple plausible strengthenings when a variable is introduced into the typing context and then *combine* the resulting types to reason about multiple feasible paths.

Now, using `TAPP` to type `bst_gen low high`, and `TVARBASE` to type the body of the `let` gives us:

```

bst_gen : lo:{v:int |  $\tau_{\text{int}}$ }  $\rightarrow$  hi:{v:int |  $\text{lo} \leq v$ }  $\rightarrow$  [v:int tree | ...], low : {v:int |  $\tau_{\text{int}}$ },
high : [v:int |  $\text{low} \leq v$ ], y : [v:int tree | bst(v)  $\wedge$   $\forall u, \text{mem}(v, u) \implies \text{lo} < u < \text{hi}$ ] [lo  $\mapsto$  low] [hi  $\mapsto$  high]  $\vdash$ 
  y : [v:int tree | v = y]

```

Observe that `TVARBASE` types the body as: `[v:int tree | v = y]`, which is not closed. To construct a well-formed term, we need a formula equivalent to this type that accounts for the type of `y` in the current type context. The `TEQ` rule allows us to interchange formulae that are equivalent under a given type context to ensure the well-formedness of the types constructed. Unlike `TSUB`, it simply changes the form of a type's qualifiers, *without altering* the scope of feasible behaviors under the current context. In this example, such an equivalent closed type, given the binding for `y` in the type context under which the expression is being type-checked, would be:

```

let (y: int tree) = bst_gen low high in y :
  [v:int tree |  $\exists y, (\text{bst}(y) \wedge \forall u, \text{mem}(y, u) \implies \text{low} < u < \text{high}) \wedge v = y$ ]

```

With these pieces in hand, we can see that the typing rule for `match` is a straightforward adaptation of the components we have already seen, where the type of matched variable v is assumed to have been strengthened by the rule `TSUB` to fit the type required to take the i th branch $\Gamma, \bar{y}:\tau_{\bar{y}} \vdash d_i(\bar{y}) : \tau_v$. We can also safely assume the type of the branch τ_i is closed under original type context Γ , relying on `TEQ` to meet this requirement. While `TMATCH` only allows for a single branch to be typechecked, applying `TMERGE` allows us to reason about the coverage provided by multiple branches, which have all been typed according to this rule.

The typing rule for recursive functions is similarly standard,⁷ with the caveat that it can only type terminating functions; since types in our language serve as witnesses to feasible executions, the result type of any recursive procedure must characterize the set of values the procedure can plausibly return. Thus, the `TFIX` rule forces its first argument to always decrease according to some well-founded relation $<$. To see why we impose this restriction, consider the function `loop`:

```
let rec loop (n: nat) = loop n
```

Without our termination check, this function can be assigned the type `{v:nat |  $\tau_{\text{nat}}$ }  $\rightarrow$  [v:nat | v = 3]`, despite the fact that this function never returns 3— or any value at all! The body of this expression can be type-checked under the following type context (via `TFIX` and `TFUN`):

```
n:{v:nat |  $\tau_{\text{nat}}$ }, loop:(n:{v:nat |  $\tau_{\text{nat}}$ }  $\rightarrow$  [v:nat | v = 3])  $\vdash$  loop n : [v:nat | v = 3]
```

⁷As in `TFUN`, the self-reference to f and the parameter of the lambda abstraction x in the recursive function body must have type annotations consistent with the basic type of the fix expression.

This judgment reflects an infinitely looping execution, however. Indeed, the same reasoning allows us to type this function with any result type. Constraining loop's argument type to be decreasing according to $<$ yields the following typing obligation:

$$n:\{v:\text{nat} \mid \top_{\text{nat}}\}, \text{loop}:(n:\{v:\text{nat} \mid v < n\} \rightarrow [v:\text{nat} \mid v = 3]) \vdash n : [v:\text{nat} \mid v = n]$$

where the qualifiers $v < n$ and $v = n$ conflict, raising a type error, and preventing loop from being recursively applied to n .

5 TYPE SOUNDNESS

Type Denotations. Assuming a standard typing judgement for basic types, $\emptyset \vdash_t e : t$, a type denotation for a type τ , $\llbracket \tau \rrbracket$, is a set of closed expressions:

$$\begin{aligned} \llbracket \{v:b \mid \phi\} \rrbracket &\doteq \{v \mid \emptyset \vdash_t v : b \wedge \phi[v \mapsto v]\} \\ \llbracket [v:b \mid \phi] \rrbracket &\doteq \{e \mid \emptyset \vdash_t e : b \wedge \forall v:b, \phi[v \mapsto v] \implies e \hookrightarrow^* v\} \\ \llbracket x:\tau_x \rightarrow \tau \rrbracket &\doteq \{f \mid \emptyset \vdash_t f : [\tau_x \rightarrow \tau] \wedge \forall v_x \in \llbracket \tau_x \rrbracket \implies f v_x \in \llbracket \tau[x \mapsto v_x] \rrbracket\} \end{aligned}$$

In the case of an overapproximate refinement type, $\{v:b \mid \phi\}$, the denotation is simply the set of all values of type b whose elements satisfy the type's refinement predicate (ϕ), when substituted for all free occurrences of v in ϕ .⁸ Dually, the denotation of an underapproximate coverage type is the set of expressions that evaluate to v whenever $\phi[v \mapsto v]$ holds, where ϕ is the type's refinement predicate. Thus, every expression in such a denotation serves as a witness to a feasible, type-correct, execution. The denotation for a function type is defined in terms of the denotations of the function's argument and result in the usual way, ensuring that our type denotation is a logical predicate.

Type Denotations under a Type Context. The denotation of a refinement type τ under a type context Γ (written $\llbracket \tau \rrbracket_\Gamma$) is:^{9,10}

$$\begin{aligned} \llbracket \tau \rrbracket_\emptyset &\doteq \llbracket \tau \rrbracket \\ \llbracket \tau \rrbracket_{x:\tau_x, \Gamma} &\doteq \{e \mid \forall v_x \in \llbracket \tau_x \rrbracket, \text{let } x = v_x \text{ in } e \in \llbracket \tau[x \mapsto v_x] \rrbracket_{\Gamma[x \mapsto v_x]}\} && \text{if } \tau \equiv \{v:b \mid \phi\} \\ \llbracket \tau \rrbracket_{x:\tau_x, \Gamma} &\doteq \{e \mid \exists \hat{e}_x \in \llbracket \tau_x \rrbracket, \forall e_x \in \llbracket \tau_x \rrbracket, \text{let } x = e_x \text{ in } e \in \bigcap_{\hat{e}_x \hookrightarrow^* v_x} \llbracket \tau[x \mapsto v_x] \rrbracket_{\Gamma[x \mapsto v_x]}\} && \text{otherwise} \end{aligned}$$

The denotation of an overapproximate refinement type under a type context is mostly unsurprising, other than our presentation choice to use a let-binding, rather than substitution, to construct the expressions included in the denotations. For a coverage type, however, the definition precisely captures our notion of a reachability witness by explicitly constructing an execution path as a sequence of let-bindings that justifies the inhabitant of the target type τ . Using let-bindings forces expressions in the denotation to make consistent choices when evaluated. The existential introduced in the definition captures the notion of an underapproximation, while the use of set intersection allows us to reason about non-determinism introduced by primitive generators like `nat_gen()`.

⁸The denotation of an overapproximate refinement type is more generally $\{e:b \mid \emptyset \vdash e : b \wedge \forall v:b, e \hookrightarrow^* v \implies \phi[x \mapsto v]\}$. However, because such types are only used for function parameters, and our language syntax only admits values as arguments, our denotation uses the simpler form.

⁹In the last case, since $\hat{e}_x$ may nondeterministically reduce to multiple values, we employ intersection (not union), similar to the DISJUNCTION rule.

¹⁰When reasoning about a subset relation between the denotations of two types under a type context $\llbracket [v:b \mid \phi_1] \rrbracket_\Gamma \subseteq \llbracket [v:b \mid \phi_2] \rrbracket_\Gamma$ we require that the denotations be computed using the same Γ ; details are provided in the full version of the paper [Zhou et al. 2023a].

Example 5.1. The term $x+1$ is included in the denotation of the type $[v:\text{nat} \mid v = x + 1 \vee v = x + x]$ under the type context $x : [v:\text{nat} \mid v = 1]$. This is justified by picking 1 for $\hat{e}_x$, which yields a set intersection that is equivalent to $[[v:\text{nat} \mid v = 2]]$. Observe that *any* expression in $[[v:\text{nat} \mid v = 1]]$, e.g. $0 \oplus 1$ and $1 \oplus 2$, yields an expression, $\text{let } x = 0 \oplus 1 \text{ in } x + 1$ or $\text{let } x = 1 \oplus 2 \text{ in } x + 1$, included in this intersection.

Example 5.2. On the other hand, the term x is not a member of this denotation. To see why, let us pick $\text{nat_gen}()$ for $\hat{e}_x$. This yields a set intersection that is equivalent to $[[v:\text{nat} \mid \top_{\text{nat}}]]$. While specific choices for e_x , e.g., $\text{nat_gen}()$, are included in this denotation, it does not work for all terms $e_x \in [[v:\text{nat} \mid v = 1]]$. As one example, $0 \oplus 1 \oplus 2$ is an element of this set, but $\text{let } x = 0 \oplus 1 \oplus 2 \text{ in } x$ is clearly not a member of $[[v:\text{nat} \mid \top_{\text{nat}}]]$. Suppose instead that we picked a more restrictive expression for $\hat{e}_x$, like the literal 1 from the previous example. Here, it is easy to choose $e_x \in [[v:\text{nat} \mid v = 1]]$ (e.g., the literal 1) such that $\text{let } x = e_x \text{ in } x \notin [[v:\text{nat} \mid v = 2]]$.

Our main soundness result establishes the correctness of type-checking in the presence of coverage types with respect to a type's denotation:

THEOREM 5.3. [Type Soundness] *For all type contexts Γ , terms e and coverage types τ , $\Gamma \vdash e : \tau \implies e \in [[\tau]]_{\Gamma}$.*

It immediately follows that a closed input generator e with coverage type $[v:b \mid \phi]$ must produce every value satisfying ϕ , as desired.

6 TYPING ALGORITHM

The declarative typing rules are highly nondeterministic, relying on a combination of the **TMERGE** and **TSUB** rules to both explore and combine the executions needed to establish the desired coverage properties. In addition, each of the auxiliary typing relations depend on logical properties of the semantic interpretation of types. Any effective type checking algorithm based on these rules must address both of these issues. Our solution to the first problem is to implement a bidirectional type checker [Dunfield and Krishnaswami 2021] whose type synthesis phase characterizes a set of feasible paths and whose type checking phase ensures those paths produce the desired results. Our solution to the second is to encode the logical properties into a *decidable* fragment of first order logic that can be effectively discharged by an SMT solver.

6.1 Bidirectional Typing Algorithm

Type Synthesis	$\Gamma \vdash e \Rightarrow \tau$	Type Check	$\Gamma \vdash e \Leftarrow \tau$
$\frac{\forall i, \text{Ty}(d_i) = \overline{y:\{v:b_y \mid \theta_y\}} \rightarrow [v:b \mid \psi_i] \quad \Gamma'_i = \overline{y:[v:b_y \mid \theta_y], a:[v:b \mid v = v_a \wedge \psi_i]} \quad \Gamma, \Gamma'_i \vdash e_i \Rightarrow \tau_i \quad \tau'_i = \text{Ex}(\Gamma'_i, \tau_i) \quad \Gamma \vdash \text{Disj}(\overline{\tau'_i}) <: \tau' \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash \text{match } v_a \text{ with } \overline{d_i \bar{y} \rightarrow e_i} \Leftarrow \tau'} \quad \text{CHKMATCH}$			
$\frac{\begin{array}{l} \Gamma \vdash v_1 \Rightarrow (a:\tau_a \rightarrow \tau_b) \rightarrow \tau_x \\ \Gamma \vdash v_2 \Leftarrow a:\tau_a \rightarrow \tau_b \quad \Gamma' = x:\tau_x \\ \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \text{Ex}(\Gamma', \tau) \\ \Gamma \vdash^{\text{WF}} \tau' \end{array}}{\Gamma \vdash \text{let } x = v_1 \ v_2 \text{ in } e \Rightarrow \tau'} \quad \text{SYNAFFUN} \quad \frac{\begin{array}{l} \Gamma \vdash v_1 \Rightarrow a:\{v:b \mid \phi\} \rightarrow \tau_x \\ \Gamma' = a:[v:b \mid v = v_2 \wedge \phi], x:\tau_x \\ \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \text{Ex}(\Gamma', \tau) \\ \Gamma \vdash^{\text{WF}} \tau' \end{array}}{\Gamma \vdash \text{let } x = v_1 \ v_2 \text{ in } e \Rightarrow \tau'} \quad \text{SYNAFFBASE}$			

Fig. 7. Selected Bidirectional Typing Rules

As is standard in bidirectional type systems, our typing algorithm consists of a type synthesis judgement ($\Gamma \vdash e \Rightarrow \tau$) and a type checking judgement ($\Gamma \vdash e \Leftarrow \tau$). Figure 7 presents the key rules for both.

Typing match. As we saw in Section 4, applying the declarative typing rule for match expressions typically requires first using several other rules to get things into the right form: TMERGE is required to analyze and combine the types of each branch, TSUB is used to equip each branch with the right typing context, and TEQ is used to remove any local or pattern variables from the type of a branch. Our bidirectional type system combines all of these into the single CHKMATCH rule shown in Figure 7. At a high level, this rule synthesizes a type for all the branches and then ensures that, in combination, they cover the desired type.

Similarly to other refinement type systems, when synthesizing the type for the branch for constructor d_i , we use a ghost variable $a: [v:b \mid v = v_a \wedge \psi_i]$ to ensure that the types of any pattern variables $\bar{y}$ are consistent with the parameters of d_i . This strategy allows us to avoid having to apply TSUB to focus on a particular branch: instead, we simply infer a type for *each* branch, and then combine them using our disjunction operation. In order for the inferred type of a branch to make sense, we need to remove any occurrences of pattern variables or the ghost variable a . To do, we use the **Ex** function, which intuitively allows us to embed information from the typing context into a type. This function takes as input a typing context Γ and type τ and produces an equivalent type $\tau <: \tau' <: \tau$ in which pattern and ghost variables do not appear free. Finally, CHKMATCH uses **Disj** to ensure that the combination of the types of all the branches cover the required type $\Gamma \vdash \text{Disj}(\tau'_i) <: \tau$.

Example 6.1. Consider how we might check that the body of the generator for natural numbers introduced in Section 2 has the expected type $[v:\text{int} \mid v \bmod 2 = 0]$.¹¹

```
int_gen:{v:unit | T_unit} → [v:int | T_int] ⊢
let (n: int) = int_gen() in let (b: bool) = n mod 2 == 0 in
match b with true → err | false → n <= [v:int | v mod 2 = 0]
```

Our typing algorithm first adds the local variable n and b to the type context, and then checks the pattern-matching expression against the given type:

```
int_gen:{v:unit | T_unit} → [v:int | T_int], n:[v:int | T_int], b:[v:bool | v ⇐⇒ n mod 2 = 0] ⊢
match b with true → err | false → n <= [v:int | v mod 2 = 0]
```

The CHKMATCH rule first synthesizes types for the two branches separately. Inferring a type of the first branch using the existing type context:

```
..., b:[v:bool | v ⇐⇒ n mod 2 = 0], b':[v:bool | v = b ∧ v] ⊢ err ⇒ [v:int | ⊥]
```

adds a ghost variable b' to reflect the fact that n must be less than 0 in this branch. By next applying the TERR rule, our algorithm infers the type $[v:\text{int} \mid \perp]$ for this branch. The rule next uses **Ex** to manifest b' in the inferred type, encoding the path constraints under which this type holds (i.e. b is true).

```
..., b:[v:bool | v ⇐⇒ n mod 2 = 0], b':[v:bool | v = b ∧ v] ⊢ err ⇒ [v:int | ∃b', b' = b ∧ b' ∧ ⊥]
```

Thus, the synthesized type for the first branch is $[v:\text{int} \mid b \wedge \perp]$ after trivial simplification. The type of the second branch provides a better demonstration of why **Ex** is needed:

```
..., b:[v:bool | v ⇐⇒ n mod 2 = 0], b':[v:bool | v = b ∧ ¬v] ⊢ n ⇒ [v:int | v = n]
```

After applying this operator, the inferred type is $[v:\text{int} \mid \exists b', b' = b \wedge \neg b' \wedge v = n]$; after simplification, this becomes $[v:\text{int} \mid \neg b \wedge v = n]$. The disjunction of these two types:

```
Disj([v:int | b ∧ ⊥], [v:int | ¬b ∧ v = n]) = [v:int | (b ∧ ⊥) ∨ (¬b ∧ v = n)]
```

¹¹We have replaced the `if` from the original example with a `match` expression, to be consistent with the syntax of λ^{TG} .

results in exactly the type shown in [Section 2](#). This can be then successfully checked against the target type $[v:nat \mid v \bmod 2 = 0]$.

Application. Our type synthesis rules for function application adopt a strategy similar to `CHKMATCH`'s, trying to infer the strongest type possible for an expression that uses the result of a function application. The rule for a function whose parameter is an overapproximate refinement type (`SYNAPPBASE`) is most interesting, since it has to bridge the gap with an argument that has an underapproximate coverage type. When typing e , the expression that uses the result of the function call, the rule augments the typing context with a ghost variable a . This variable records that the coverage type of the argument must overlap with the type expected by the function (both must satisfy the refinement predicate ϕ): if this intersection is empty, i.e., the type of a is equivalent to $\perp$, we will fail to infer a type for e , as no type will be well-formed in this context. As with `CHKMATCH`, `SYNAPPBASE` uses `Ex` to ensure that it does not infer a type that depends on a .

6.2 Auxiliary Typing Functions

The auxiliary `Disj` and `Ex` operations are a straightforward syntactic transformations; their full definitions can be found in the full version of the paper [[Zhou et al. 2023a](#)]. More interesting is how we check well-formedness and subtyping. Our type checking algorithm translates both obligations into logical formulae that can be discharged by a SMT solver. Both obligations are encoded by the `Query` subroutine shown in [Algorithm 1](#). `Query`($\Gamma, [v:b \mid \phi_1], [v:b \mid \phi_2]$) encodes the bindings in Γ in the typing context from right to left, before checking whether ϕ_1 implies ϕ_2 . Variables with function types, on the other hand, are omitted entirely, as qualifiers cannot have function variables in FOL. Variables with an overapproximate (underapproximate) type are translated as a universally (existential) quantified variable, and are encoded into the refinement of both coverage types.

Example 6.2. Consider the subtyping obligation generated by [Example 6.1](#) above:

$$\text{int_gen}\{v:\text{unit} \mid \tau_{\text{unit}}\} \rightarrow [v:\text{int} \mid \tau_{\text{int}}], n:[v:\text{int} \mid \tau_{\text{int}}], b:[v:\text{bool} \mid v \iff n \bmod 2 = 0] \vdash \\ [v:\text{int} \mid (b \wedge \perp) \vee (\neg b \wedge v = n)] <: [v:\text{int} \mid v \geq 0]$$

This obligation is encoded by the following call to `Query`

$$\begin{aligned} &\text{Query}(\text{int_gen}\{v:\text{unit} \mid \tau_{\text{unit}}\} \rightarrow [v:\text{int} \mid \tau_{\text{int}}], n:[v:\text{int} \mid \tau_{\text{int}}], b:[v:\text{bool} \mid v \iff n \bmod 2 = 0]) \\ &\quad [v:\text{int} \mid (b \wedge \perp) \vee (\neg b \wedge v = n)], [v:\text{int} \mid v \geq 0]) \equiv \\ &\text{Query}(\text{int_gen}\{v:\text{unit} \mid \tau_{\text{unit}}\} \rightarrow [v:\text{int} \mid \tau_{\text{int}}], n:[v:\text{int} \mid \tau_{\text{int}}], \\ &\quad [v:\text{int} \mid \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge \perp) \vee (\neg b \wedge v = n)], [v:\text{int} \mid \exists b, b \iff n \bmod 2 = 0 \wedge v \geq 0]) \equiv \\ &\text{Query}(\text{int_gen}\{v:\text{unit} \mid \tau\} \rightarrow [v:\text{int} \mid \tau_{\text{int}}], \\ &\quad [v:\text{int} \mid \exists n, \tau_{\text{int}} \wedge \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge \perp) \vee (\neg b \wedge v = n)], \\ &\quad [v:\text{int} \mid \exists n, \tau_{\text{int}} \wedge \exists b, b \iff n \bmod 2 = 0 \wedge v \geq 0]) \equiv \\ &\text{Query}(\emptyset, [v:\text{int} \mid \exists n, \tau_{\text{int}} \wedge \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge \perp) \vee (\neg b \wedge v = n)], \\ &\quad [v:\text{int} \mid \exists n, \tau_{\text{int}} \wedge \exists b, b \iff n \bmod 2 = 0 \wedge v \geq 0]) \equiv \\ &\quad \forall v, \exists n, \tau_{\text{int}} \wedge \exists b, b \iff n \bmod 2 = 0 \wedge v \geq 0 \implies \\ &\quad \exists n, \tau_{\text{int}} \wedge \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge \perp) \vee (\neg b \wedge v = n) \end{aligned}$$

This is equivalent to formula (2) from [Section 2](#):

$$\forall v, (v \geq 0) \implies (\exists n, \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge \perp) \vee (\neg b \wedge v = n))$$

Using `Query`, it is straightforward to discharge well-formedness and subtyping obligations using the rules shown in [Figure 8](#). In the case of `WFBASE`, for example, observe that the error term `err` is always an inhabitant of the type $[v:b \mid \perp]$ for arbitrary base type b . Thus, to check the last

$$\frac{\not\models \text{Query}(\Gamma, [v:b \mid \perp], [v:b \mid \phi])}{\text{err} \notin \llbracket [v:b \mid \phi] \rrbracket_{\Gamma}} \quad \frac{\models \text{Query}(\Gamma, [v:b \mid \phi_1], [v:b \mid \phi_2])}{\Gamma \vdash [v:b \mid \phi_1] <: [v:b \mid \phi_2]} \quad \frac{\models \text{Query}(\Gamma, [v:b \mid \phi_2], [v:b \mid \phi_1])}{\Gamma \vdash \{v:b \mid \phi_1\} <: \{v:b \mid \phi_2\}}$$

Fig. 8. Auxillary Typing Functions

Algorithm 1: Subtyping Query

```

1 Procedure Query(( $\Gamma, [v:b \mid \phi_1], [v:b \mid \phi_2]$ )) :=
2   match  $\Gamma$ :
3     case  $\emptyset$  do
4       return  $\forall v:b, \phi_2 \implies \phi_1$ ;
5     case  $\Gamma, x:(a:\tau_a \rightarrow \tau)$  do
6        $\phi \leftarrow \text{Query}((\Gamma, [v:b \mid \phi_1], [v:b \mid \phi_2]));$ 
7       return  $\phi$ ;
8   case  $\Gamma, x:\{v:b_x \mid \phi_x\}$  do
9      $\tau_1 \leftarrow [v:b \mid \forall x:b_x, \phi_x[v \mapsto x] \implies \phi_1];$ 
10     $\tau_2 \leftarrow [v:b \mid \forall x:b_x, \phi_x[v \mapsto x] \implies \phi_2];$ 
11    return Query( $\Gamma, [v:b \mid \tau_1], [v:b \mid \tau_2]$ );
12  case  $\Gamma, x:[v:b_x \mid \phi_x]$  do
13     $\tau_1 \leftarrow [v:b \mid \exists x:b_x, \phi_x[v \mapsto x] \wedge \phi_1];$ 
14     $\tau_2 \leftarrow [v:b \mid \exists x:b_x, \phi_x[v \mapsto x] \wedge \phi_2];$ 
15    return Query( $\Gamma, [v:b \mid \tau_1], [v:b \mid \tau_2]$ );

```

assumption of WFBASE, it suffices to iteratively check if any coverage types in the type context are a supertype of their associated bottom type.

Discharging subtyping obligations is slightly more involved, as we need to ensure that the formulas sent to the SMT solver are decidable. Observe that in order to produce effectively decidable formulas, the encoding strategy realized by `Query` always generates a formula of the form $\forall \bar{x}. \exists \bar{y}. \phi$, i.e. it does not allow for arbitrary quantifier alternations. To ensure that this is sound strategy, we restrict all overapproximate refinement types in a type context to not have any free variables that have a coverage type. This constraint allows us to safely lift all universal quantifiers to the top level, thus avoiding arbitrary quantifier alternations.

As an example of a scenario disallowed by this restriction, consider the following type checking judgment:

$$x:[v:\text{nat} \mid v > 0] \vdash \lambda y : \text{nat}. x + y \Leftarrow y:\{v:\text{nat} \mid v > x + 1\} \rightarrow [v:\text{nat} \mid \phi]$$

This judgment produces the following subtyping check:

$$x:[v:\text{nat} \mid v > 0], y:\{v:\text{nat} \mid v > x + 1\} \vdash [v:\text{nat} \mid v = x + y] <: [v:\text{nat} \mid \phi]$$

where the normal refinement type $\{v:\text{nat} \mid v > x + 1\}$ in the type context has free variable x that has coverage type. Evaluating this judgment entails solving the formula:

$$\forall v, (\exists x, x > 0 \wedge (\forall y, y > x + 1 \implies \phi)) \implies (\exists x, x > 0 \wedge (\forall y, y > x + 1 \implies v = x + y))$$

which is not decidable due to the quantifier alternation $\forall v \exists x \forall y$.

THEOREM 6.3. [Soundness of Algorithmic Typing] For all type context Γ , term e and coverage type τ , $\Gamma \vdash e \Leftarrow \tau \implies \Gamma \vdash e : \tau$

THEOREM 6.4. [Completeness of Algorithmic Typing] Assume an oracle for all formulas produced by the `Query` subroutine. Then for any type context Γ , term e and coverage type τ , $\Gamma \vdash e : \tau \implies \Gamma \vdash e \Leftarrow \tau$.

7 EVALUATION

Implementation. We have implemented a coverage type checker, called Poirot, based on the above approach. Poirot targets functional, non-concurrent OCaml programs that rely on libraries

Table 3. Experimental results on hand-written generators. Existing benchmarks are annotated with their source: QuickChick [Lampropoulos and Pierce 2022] (*), QuickCheck [Claessen and Hughes 2000] (°), and bespoke test input generators from [Lampropoulos et al. 2019] (*) and [Zhou et al. 2021] (°).

	#Branch	Recursive	#LocalVar	#MP	#Query	(max. #V, #E)	total (avg. time)(s)
SizedList*	4	✓	12	2	11	(7, 9)	0.35(0.03)
SortedList*	4	✓	11	4	13	(9, 9)	6.77(0.52)
UniqueList°	3	✓	8	3	10	(7, 7)	0.64(0.06)
SizedTree*	4	✓	13	2	14	(9, 12)	0.48(0.03)
CompleteTree*	3	✓	10	2	13	(8, 10)	0.38(0.03)
RedBlackTree*	6	✓	36	3	70	(16, 53)	6.69(0.10)
SizedBST*	5	✓	20	4	29	(17, 18)	12.20(0.42)
BatchedQueue°	2		6	1	9	(7, 5)	0.52(0.06)
BankersQueue°	2		6	1	11	(7, 6)	0.46(0.04)
Stream°	4		13	2	13	(8, 11)	0.44(0.03)
SizedHeap°	5	✓	16	4	18	(12, 15)	3.89(0.22)
LeftistHeap°	3	✓	11	1	16	(9, 11)	0.54(0.03)
SizedSet°	4	✓	16	4	23	(14, 15)	4.66(0.20)
UnbalanceSet°	5	✓	20	4	29	(17, 18)	9.32(0.32)

to manipulate algebraic data types; it consists of approximately 11K lines of OCaml and uses Z3 [de Moura and Bjørner 2008] as its backend solver.

Poirot takes as input an OCaml program representing a test input generator and a user-supplied coverage type for that generator. After basic type-checking and translation into MNF, Poirot applies bidirectional type inference and checking to validate that the program satisfies the requirements specified by the type. Our implementation provides built-in coverage types for a number of OCaml primitives, including constants, various arithmetic operators, and data constructors for a range of datatypes. Refinements defined in coverage types can also use predefined (polymorphic) method predicates that capture non-trivial datatype shape properties. For example, the method predicate $mem(1, u)$ indicates the element $u:b$ is contained in the data type instance $1:b$ T ; the method predicate $len(1, 3)$ indicates the list 1 has length 3, or the tree 1 has depth 3. The semantics of these method predicates are defined as a set of FOL-encoded lemmas and axioms to facilitate automated verification; e.g., the lemma $len(1, 0) \implies \forall u, \neg mem(1, u)$ indicates that the empty datatype instance contains no element.

7.1 Completeness of Hand-Written Generators

We have evaluated Poirot on a corpus of hand-written, non-trivial test input generators drawn from a variety of sources (see Table 3). These benchmarks provide test input generators over a diverse range of datatypes, including various kinds of lists, trees, queues, streams, heaps, and sets. For each datatype implementation, Poirot type checks the provided implementation against its supplied coverage type to verify that the generator is able to generate all possible datatype instances consistent with this type. Our method predicates allow us capture non-trivial structural properties. For example, to verify a red-black tree generator, we use the predicate $black_height(v, n)$ to indicate that all branches of the tree v have exactly n black nodes, the predicate $no_red_red(v)$ to indicate v contains no red node with red children, and the predicate $root_color(v, b)$ to indicate the root of the tree v has the red (black) color when the boolean value b is true (false).¹²

Given this rich set of predicates, it is straightforward to express interesting coverage types. For example, given size s and lower bound lo , we can express the property that a sorted list generator

¹²These method predicates can be found in the implementation of the red-black tree generator given in [Lampropoulos and Pierce 2022].

sorted_list_gen *must* generate all possible sorted lists with the length s and in which all elements are greater than or equal to lo , as the following type:

$$s:\{v:int \mid v \leq 0\} \rightarrow lo:\{v:int \mid \top\} \rightarrow [v:int \text{ list} \mid len(v, s) \wedge sorted(v) \wedge \forall u, mem(v, u) \implies lo \leq u]$$

Notice that this type is remarkably similar to a normal refinement type:

$$s:\{v:int \mid v \leq 0\} \rightarrow lo:\{v:int \mid \top\} \rightarrow \{v:int \text{ list} \mid len(v, s) \wedge sorted(v) \wedge \forall u, mem(v, u) \implies lo \leq u\}$$

albeit with the return type marked as a coverage type to capture our desired must-property.

The first group of columns in Table 3 describes the salient features of our benchmarks. Each benchmark exhibits non-trivial control-flow, containing anywhere from 2 to 6 nested branches; the majority of our benchmarks are also recursive (column Recursive). The number of local (i.e., let-bound) variables (column #LocalVars) is a proxy for path lengths that must be encoded within the types inferred by our type-checker; column #MP indicates the number of method predicates found in the benchmark's type specification.

The second group of columns presents type checking results. Column #Query indicates the number of SMT queries that are triggered during type checking. Column $\#(\forall, \exists)$ indicates the maximum number of universal and existential quantifiers in these queries, respectively. The $\exists$ column is a direct reflection of control-flow (path) complexity – complex generators with deeply nested match-expressions like RedBlackTree result in queries with over 50 existential quantifiers. These numbers broadly track with the values in columns #Branch and #LocalVar. Despite the complexity of some of these queries, as evidenced by the number of their quantifiers, overall verification time (average verification time per query, resp.), reported in the last column, is quite reasonable, with times ranging from .35 to 12.20 seconds, with more than half of the benchmarks finishing in less than a second.

7.2 Case Study: Well-Typed STLC Terms

We have also applied Poirot to a more substantial example: a generator for well-typed simply typed lambda calculus (STLC) terms in the vein of Lampropoulos et al. [2017]; Pałka et al. [2011]. Such a generator can be used to test that the typing relation guarantees the expected runtime behaviors of programs, e.g. progress and preservation. In addition to the complexity of the coverage property itself (well-typedness), this case study features multiple inductive datatypes (for types, terms, and typing contexts, as shown in Figure 9), and 13 auxiliary functions. The coverage type of gen_term_size, the top-level generator, stipulates that it can generate all terms of a desired type, up to a user-provided size bound:

```
type ty = Ty_nat
        | Ty_arr of ty * ty

type term =
  | Const of int
  | Var of int
  | Abs of ty * term
  | App of term * term

type tyctx = ty list
```

Fig. 9. Datatypes from the STLC case study.

$$\text{gen_term_size} : \underbrace{n:\{v:int \mid 0 \leq v\}}_{\text{maximum term size}} \rightarrow \underbrace{t:\{v:ty \mid \top\}}_{\text{type of term}} \rightarrow \underbrace{\Gamma:\{v:tyctx \mid \top\}}_{\text{typing context}} \rightarrow \underbrace{[v:term \mid has_ty \Gamma \ v \ t \wedge max_app_num \ v \ n]}_{\text{result is well-typed and has at most } n \text{ applications}}$$

The results of using Poirot to verify that gen_term_size meets the above specification are shown in Table 4. The table also reports the results for the most interesting auxiliary functions used by the function. The last column shows that Poirot is able to verify these functions within a reasonable time, ranging from 0.47 to 149.39 seconds. Although more complex functions (as indicated by the column labeled #Branch) require more time to verify, total verification time is nonetheless reasonable: 198.43 seconds in total. Taken together, these results highlight the compositionality of Poirot's type-based approach: each of the 13 auxiliary functions used by gen_term_size is individually

Table 4. Experimental results from the STLC case study. Each function is implemented as a wrapper around a subsidiary function that takes an additional strictly decreasing argument to ensure termination (the original QuickChick implementations uses Coq’s Program command for this purpose). These subsidiary functions are responsible for the bulk of the computation, so we report the results for those functions here.

	#Branch	Recursive	#LocalVar	#MP	#Query	(max. # $\forall$, # $\exists$)	total (avg. time)(s)
type_eq	6	✓	7	5	9	(10, 9)	36.26(4.03)
gen_type	3	✓	10	1	15	(10, 12)	0.47(0.03)
var_with_type	5	✓	12	7	13	(12, 8)	7.77(0.60)
gen_term_no_app	3	✓	13	10	20	(14, 15)	4.37(0.22)
gen_term_size	4	✓	24	9	50	(27, 27)	148.39(2.97)

type-checked against its signature; these signatures are then used to verify any procedures that call the function.

Interestingly, the function `type_eq` has a longer average query time than all other functions, despite having fewer local variables and method predicates. This function implements a deterministic equality test, returning `true` when two types are the same and `false` otherwise. Thus, the coverage type of this function degenerates into a singleton type for each of the branches, resulting in stricter queries to the SMT solver that take longer to find a valid witness.

Discussion. To handle the complexity of this benchmark, Poirot requires 14 method predicates and 35 axioms, the large majority of which correspond to helper definitions and lemmas from the original development. The predicates that encode typing and the bounds on the number of applications in a term (*has_ty* and *max_num_app*, resp.) come directly from the QuickCheck version, for example. The following axiom encodes the semantic relationship of these predicates

$$\forall(\Gamma: \text{tyctx}) (t: \text{ty}) (e: \text{term}), \text{has_ty } \Gamma \ e \ t \iff (\exists(n: \text{nat}), \text{max_num_app } e \ n \wedge \text{has_ty } \Gamma \ e \ t)$$

and is analogous to the helper lemma `has_ty_max_tau_correct` in the Coq development. In addition, some predicates and axioms are independent of this particular case study: the typing context is implemented as a list of STLC types, and thus we were able to reuse generic predicates and axioms about polymorphic lists.

7.3 Completeness of Synthesized Generators

Table 5. Quantifying the space of safe and complete test input generators constructed by an automated program synthesis tool.

Benchmark	#Total	#Complete
UniqueList	284	10
SizedList	126	28
SortedList	30	8
SizedTree	103	2
SizedBST	229	54
RedBlackTree	234	2

An underlying hypothesis motivating our work is that writing sound and *complete* test input generators can be subtle and tricky, as demonstrated by our motivating example (Figure 1). To justify this hypothesis, we repurposed an existing deductive component-based program synthesizer [Mishra and Jagannathan 2022] to automatically synthesize correct (albeit possibly incomplete) generators that satisfy a specification given as an overapproximate refinement type; these generators are then fed to Poirot to validate their completeness. We provided the synthesizer with a datatype definition and a set of specifications describing constraints on that datatype the synthesized generator should use, along with a library of functions, including primitive generators such as `nat_gen`, available to the synthesizer for construction. A refinement type-guided enumeration is performed to find all correct programs consistent with the specification. Since the space of these programs is potentially quite large (possibly infinite), we constrain the synthesizer to only generate programs with bounded function call depths; in our experiments, this bound was set to three. The generator outputs all programs that are safe with respect to the specification. Table 5 shows results of this experiment for five of the benchmarks given in Table 3; results for the other benchmarks are similar. We report the total number of

constraints on that datatype the synthesized generator should use, along with a library of functions, including primitive generators such as `nat_gen`, available to the synthesizer for construction. A refinement type-guided enumeration is performed to find all correct programs consistent with the specification. Since the space of these programs is potentially quite large (possibly infinite), we constrain the synthesizer to only generate programs with bounded function call depths; in our experiments, this bound was set to three. The generator outputs all programs that are safe with respect to the specification. Table 5 shows results of this experiment for five of the benchmarks given in Table 3; results for the other benchmarks are similar. We report the total number of


```

1  let rec sized_list_gen
2    (size : int) : (int list) =
3    if (size == 0) then []
4    else
5      if (bool_gen ()) then
6        sized_list_gen (size - 1)
7      else
8        int_gen () ::
9          (sized_list_gen (size - 1))

```

```

1  let rec sized_list_gen
2    (size : int) : (int list) =
3    if (size == 0) then []
4    else
5      int_gen () ::
6        (sized_list_gen (size - 1))

```

```

1  let rec sized_list_gen
2    (size : int) : (int list) =
3    if (size == 0) then []
4    else
5      if (bool_gen ()) then
6        sized_list_gen (size - 1)
7      else
8        size ::
9          (sized_list_gen (size - 1))

```

(a) A sound and complete generator. (b) A sound but incomplete generator. (c) Another sound but incomplete generator.

Fig. 10. Three example generators that generate size-bounded lists.

synthesized generators (#Total) constructed and the number of those that are *correct and complete* as verified by Poirot (#Complete). The table confirms our hypothesis that the space of complete generators with respect to the supplied coverage type is significantly smaller than the space of safe generators, as defined by an overapproximate refinement type specification.

More concretely, Figure 10 shows three synthesized generators that satisfy the following specification of a list generator that is meant to construct *all* lists no longer than some provided bound:

$$\text{size}:\{v:\text{int} \mid v \leq 0\} \rightarrow [v:\text{int list} \mid \forall u, \text{len}(v, u) \implies (0 \leq u \wedge u \leq \text{size})]$$

Figure 10b is incomplete because it never generates an empty list when the size parameter *size* is greater than 0. On the other hand, while Figure 10c does generate empty lists, the *else* branch of its second conditional has a fixed first element and will therefore never generate lists with distinct elements. The complete generator shown in Figure 10a incorporates a control-flow path (line 5) that can non-deterministically choose to make a recursive call to *sized_list_gen* with a smaller size, thereby allowing it to generate lists of variable size up to the size bound, including the empty list; another conditional branch uses *int_gen*() to generate a new randomly selected list element, thereby allowing the implementation to generate lists containing distinct elements. We again emphasize that Poirot was able to verify the correct generator and discard the two incorrect generators *automatically*, without any user involvement.

8 RELATED WORK

The effectiveness of PBT suffers when the property of interest has a strict precondition [Lampropoulos 2018], because most of the inputs produced by a purely random test generation strategy will be simply discarded. As a result, there has been much recent interest on improving the coverage of test generators with respect to a particular precondition. Proposed solutions range from adopting ideas from fuzzing [Dolan 2022; Zalewski 2020] to intelligently mutate the outputs produced by the generator [Lampropoulos et al. 2019; Padhye et al. 2019], to focusing on generators for particular classes of inputs (e.g., well-typed programs) [Fetscher et al. 2015; Palka et al. 2011; Yang et al. 2011], to automatically building complete-by-construction generators [Claessen et al. 2014; Lampropoulos et al. 2017, 2018]. While sharing broadly similar goals with these proposals, our approach differs significantly in its framing of coverage in purely type-theoretic terms. This fundamental change in perspective allows us to statically and compositionally verify coverage properties of a generator without the need for any form of instrumentation on, or runtime monitoring of, the program under test (as in [Dolan 2022; Lampropoulos et al. 2019]). Unlike other approaches that have also considered the verification of a generator’s coverage properties [Dybjer et al. 2003, 2004; Paraskevopoulou et al. 2015] using a mechanized proof assistant, our proposed type-based framing is highly-automated and inherently compositional. Expressing coverage as part of a type system also allows us to be agnostic to (a) how generators are constructed, (b) the particulars of the application domain [Fetscher et al. 2015; Palka et al. 2011; Yang et al. 2011], and (c) the specific structure of the properties being tested [Lampropoulos et al. 2017, 2018]. Poirot’s ability to specify

and type-check a complex coverage property depends only on whether we can express a desired specification using available method predicates.

A number of logics have been proposed for reasoning about underapproximations of program behavior, including the recently developed incorrectness logic (IL) [O’Hearn 2019; Raad et al. 2020], reverse Hoare logic (RHL) [de Vries and Koutavas 2011], and dynamic logic (DL) [Pratt 1976]. Both IL and RHL are formalisms similar to Hoare logic, but support composable specifications that assert underapproximate postconditions, with IL adding special post-assertions for error states. IL was originally proposed as a way of formalizing the conditions under which a particular program point (say an error state) is guaranteed to be reachable, and has recently been used in program analyses that discover memory errors [Le et al. 2022]. DL, in contrast, reinterprets Hoare logic as a multi-modal logic equipped with operators for reasoning about the existence of executions that end in a state satisfying some desired postcondition. This paper instead provides the first development that interprets these notions in the context of a type system for a rich functional language. While our ideas are formulated in the context of verifying coverage properties for test input generators, we believe our framework can be equally adept in expressing type-based program analyses for bug finding or compiler optimizations.

Our focus on reasoning about coverage properties of test input generators distinguishes our approach, in obvious ways from other refinement type-based testing solutions such as TARGET [Seidel et al. 2015]. Nonetheless, our setup follows the same general verification playbook as Liquid Types [Jhala and Vazou 2021; Vazou et al. 2014] — our underapproximate specifications are identical to their overapproximate counterpart, except that we syntactically distinguish the return types for functions to reflect their expected underapproximate (rather than overapproximate) behavior. An important consequence of this design is that the burden of specifying and checking the coverage behavior of a program is no greater than specifying its safety properties.

Another related line of work has explored how to reason about the distribution of data produced by a function [Albarghouthi et al. 2017; Bastani et al. 2019], with a focus on ensuring that these distributions are free of unwanted biases. These works have considered decision-making and machine-learning applications, in which these sorts of fairness properties can be naturally encoded as (probabilistic) formulas in real arithmetic. In contrast, coverage types can only verify that a generator has a nonzero probability of producing a particular output. Extending our type system and its guarantees to provide stronger fairness guarantees about the distribution of the sorts of discrete data produced by test input generators is an exciting direction for future work.

9 CONCLUSION

This paper adapts principles of underapproximate reasoning found in recent work on Incorrectness Logic to the specification and automated verification of test input generators used in modern property-based testing systems. Specifications are expressed in the language of refinement types, augmented with coverage types, types that reflect underapproximate constraints on program behavior. A novel bidirectional type-checking algorithm enables an expressive form of inference over these types. Our experimental results demonstrate that our approach is capable of verifying both sophisticated hand-written generators, as well as being able to successfully identify type-correct (in an overapproximate sense) but coverage-incomplete generators produced from a deductive refinement type-aware synthesizer.

ACKNOWLEDGEMENTS

We thank our shepherd Leonidas Lampropoulos and the anonymous reviewers for their detailed comments and suggestions. This work was supported in part by DARPA grants HR0011-19-C-0073 and HR0011-22-90062, and the Department of Defense under grant FA8649-21-P-1511.

10 DATA AVAILABILITY

An artifact containing our implementation, benchmark suite, results and corresponding Coq proofs is publicly available on Zenodo [Zhou et al. 2023b].

REFERENCES

- Aws Albarghouthi, Loris D’Antoni, Samuel Drews, and Aditya V. Nori. 2017. FairSquare: Probabilistic Verification of Program Fairness. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 80 (oct 2017), 30 pages. <https://doi.org/10.1145/3133904>
- Osbert Bastani, Xin Zhang, and Armando Solar-Lezama. 2019. Probabilistic Verification of Fairness Properties via Concentration. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 118 (Oct 2019), 27 pages. <https://doi.org/10.1145/3360544>
- Koen Claessen. 2020. *QuickCheck*. <https://hackage.haskell.org/package/QuickCheck>
- Koen Claessen, Jonas Duregård, and Michal H. Palka. 2014. Generating Constrained Random Data with Uniform Distribution. *Journal of Functional Programming* 25. https://doi.org/10.1007/978-3-319-07151-0_2
- Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP ’00)*. Association for Computing Machinery, New York, NY, USA, 268–279. <https://doi.org/10.1145/351240.351266>
- Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
- Edsko de Vries and Vasileios Koutavas. 2011. Reverse Hoare Logic. In *Software Engineering and Formal Methods*, Gilles Barthe, Alberto Pardo, and Gerardo Schneider (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 155–171. https://doi.org/10.1007/978-3-642-24690-6_12
- Stephen Dolan. 2022. *Crowbar*. <https://github.com/stedolan/crowbar>
- Jana Dunfield and Neel Krishnaswami. 2021. Bidirectional Typing. *ACM Comput. Surv.* 54, 5, Article 98 (may 2021), 38 pages. <https://doi.org/10.1145/3450952>
- Peter Dybjer, Qiao Haiyan, and Makoto Takeyama. 2003. Combining Testing and Proving in Dependent Type Theory. In *Theorem Proving in Higher Order Logics, 16th International Conference, TPHOLs 2003, Rom, Italy, September 8-12, 2003, Proceedings (Lecture Notes in Computer Science, Vol. 2758)*, David A. Basin and Burkhart Wolff (Eds.). Springer, 188–203. https://doi.org/10.1007/10930755_12
- Peter Dybjer, Qiao Haiyan, and Makoto Takeyama. 2004. Random Generators for Dependent Types. In *Theoretical Aspects of Computing - ICTAC 2004, First International Colloquium, Guiyang, China, September 20-24, 2004, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 3407)*, Zhiming Liu and Keijiro Araki (Eds.). Springer, 341–355. https://doi.org/10.1007/978-3-540-31862-0_25
- FastCheck 2022. *fast-check: Property based testing for JavaScript and TypeScript*. <https://dubzzz.github.io/fast-check.github.io/>
- Burke Fetscher, Koen Claessen, Michal H. Palka, John Hughes, and Robert Bruce Findler. 2015. Making Random Judgments: Automatically Generating Well-Typed Terms from the Definition of a Type-System. In *Programming Languages and Systems - 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings (Lecture Notes in Computer Science, Vol. 9032)*, Jan Vitek (Ed.). Springer, 383–405. https://doi.org/10.1007/978-3-662-46669-8_16
- Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. The Essence of Compiling with Continuations. In *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation (Albuquerque, New Mexico, USA) (PLDI ’93)*. Association for Computing Machinery, New York, NY, USA, 237–247. <https://doi.org/10.1145/155090.155113>
- Kimball Germane and Jay McCarthy. 2021. Newly-single and Loving It: Improving Higher-Order Must-Alias Analysis with Heap Fragments. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–28. <https://doi.org/10.1145/3473601>
- Patrice Godefroid, Aditya V. Nori, Sriram K. Rajamani, and Sai Deep Tetali. 2010. Compositional May-Must Program Analysis: Unleashing the Power of Alternation. In *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (Madrid, Spain) (POPL ’10)*. Association for Computing Machinery, New York, NY, USA, 43–56. <https://doi.org/10.1145/1706299.1706307>
- John Hatcliff and Olivier Danvy. 1994. A Generic Account of Continuation-Passing Styles. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (Portland, Oregon, USA) (POPL ’94)*. Association for Computing Machinery, New York, NY, USA, 458–471. <https://doi.org/10.1145/174675.178053>
- C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (oct 1969), 576–580. <https://doi.org/10.1145/363235.363259>
- Hypothesis 2022. *Hypothesis*. <https://github.com/HypothesisWorks/hypothesis/tree/master/hypothesis-python>

- Suresh Jagannathan, Peter Thiemann, Stephen Weeks, and Andrew K. Wright. 1998. Single and Loving It: Must-Alias Analysis for Higher-Order Languages. In *POPL '98, Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Diego, CA, USA, January 19-21, 1998*, David B. MacQueen and Luca Cardelli (Eds.). ACM, 329–341. <https://doi.org/10.1145/268946.268973>
- Ranjit Jhala and Niki Vazou. 2021. Refinement Types: A Tutorial. *Found. Trends Program. Lang.* 6, 3-4 (2021), 159–317. <https://doi.org/10.1561/25000000032>
- Leonidas Lampropoulos. 2018. *Random Testing for Language Design*. Ph.D. Dissertation. University of Pennsylvania. <https://repository.upenn.edu/edissertations/2879>
- Leonidas Lampropoulos, Diane Gallois-Wong, Cătălin Hrițcu, John Hughes, Benjamin C. Pierce, and Li-yao Xia. 2017. Beginner's Luck: A Language for Property-Based Generators. *SIGPLAN Not.* 52, 1 (jan 2017), 114–129. <https://doi.org/10.1145/3093333.3009868>
- Leonidas Lampropoulos, Michael Hicks, and Benjamin C. Pierce. 2019. Coverage Guided, Property Based Testing. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 181 (oct 2019), 29 pages. <https://doi.org/10.1145/3360607>
- Leonidas Lampropoulos, Zoe Paraskevopoulou, and Benjamin C. Pierce. 2018. Generating Good Generators for Inductive Relations. *Proc. ACM Program. Lang.* 2, POPL (2018), 45:1–45:30. <https://doi.org/10.1145/3158133>
- Leonidas Lampropoulos and Benjamin C. Pierce. 2022. *QuickChick: Property-Based Testing in Coq*. Software Foundations, Vol. 4. Electronic textbook. Version 1.3.1, <https://softwarefoundations.cis.upenn.edu>.
- Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. 2022. Finding Real Bugs in Big Programs with Incorrectness Logic. *Proc. ACM Program. Lang.* 6, OOPSLA (2022), 1–27. <https://doi.org/10.1145/3527325>
- Ashish Mishra and Suresh Jagannathan. 2022. Specification-Guided Component-Based Synthesis from Effectful Libraries. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 147 (oct 2022), 30 pages. <https://doi.org/10.1145/3563310>
- Peter W. O'Hearn. 2019. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (dec 2019), 32 pages. <https://doi.org/10.1145/3371078>
- Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. 2019. Semantic Fuzzing with Zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, 329–340. <https://doi.org/10.1145/3293882.3330576>
- Michal H. Palka, Koen Claessen, Alejandro Russo, and John Hughes. 2011. Testing an Optimising Compiler by Generating Random Lambda Terms. In *Proceedings of the 6th International Workshop on Automation of Software Test (Waikiki, Honolulu, HI, USA) (AST '11)*. Association for Computing Machinery, New York, NY, USA, 91–97. <https://doi.org/10.1145/1982595.1982615>
- Zoe Paraskevopoulou, Cătălin Hrițcu, Maxime Dénès, Leonidas Lampropoulos, and Benjamin C. Pierce. 2015. Foundational Property-Based Testing. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 325–343. https://doi.org/10.1007/978-3-319-22102-1_22
- Vaughan R Pratt. 1976. Semantical consideration on Floyd-Hoare logic. In *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*. IEEE, 109–121.
- Azalea Raad, Josh Berdine, Hoang-Hai Dang, Derek Dreyer, Peter O'Hearn, and Jules Villard. 2020. Local Reasoning About the Presence of Bugs: Incorrectness Separation Logic. In *Computer Aided Verification: 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part II* (Los Angeles, CA, USA). Springer-Verlag, Berlin, Heidelberg, 225–252. https://doi.org/10.1007/978-3-030-53291-8_14
- RustCheck 2021. *Crate for PBT in Rust*. <https://github.com/BurntSushi/quickcheck>
- ScalaCheck 2021. *ScalaCheck*. <https://scalacheck.org/>
- Eric L. Seidel, Niki Vazou, and Ranjit Jhala. 2015. Type Targeted Testing. In *Programming Languages and Systems*. Springer Berlin Heidelberg, Berlin, Heidelberg, 812–836. https://doi.org/10.1007/978-3-662-46669-8_33
- Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement Types for Haskell. *SIGPLAN Not.* 49, 9 (aug 2014), 269–282. <https://doi.org/10.1145/2692915.2628161>
- Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) (PLDI '11). Association for Computing Machinery, New York, NY, USA, 283–294. <https://doi.org/10.1145/1993498.1993532>
- Michal Zalewski. 2020. *American Fuzzy Lop*. <https://github.com/google/aflop>
- Zhe Zhou, Robert Dickerson, Benjamin Delaware, and Suresh Jagannathan. 2021. Data-driven abductive inference of library specifications. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–29. <https://doi.org/10.1145/3485493>
- Zhe Zhou, Ashish Mishra, Benjamin Delaware, and Suresh Jagannathan. 2023a. Covering All the Bases: Type-Based Verification of Test Input Generators. arXiv:2304.03393 [cs.PL]
- Zhe Zhou, Ashish Mishra, Benjamin Delaware, and Suresh Jagannathan. 2023b. PLDI2023 Artifact: Covering All the Bases: Type-Based Verification of Test Input Generators. <https://doi.org/10.5281/zenodo.7811004>

Received 2022-11-10; accepted 2023-03-31